

FRAMEWORK VÀ CÔNG CỤ



TÓM TẮT

Phần D là tầng công cụ: thư viện phát hiện marker, các API PnP và ý nghĩa chính xác của từng cờ, hộp công cụ hiệu chuẩn, thư viện visual servoing, công cụ photogrammetry để đo constellation, và công cụ đánh giá. Nó được đặt gần cuối vì nó *hết hạn nhanh nhất* — một API đổi tên, một hàm mới xuất hiện, và chương này sai. Nếu chỉ nhớ một điều: trong toàn bộ danh sách, hàm có giá trị cao nhất cho bài toán docking là `solvePnPGeneric`, vì nó trả về *nhiều nghiệm kèm sai số tái chiếu của từng nghiệm* — tức là nó đưa thẳng cho bạn đại lượng cần để đo ambiguity, thứ mà mọi API PnP khác giấu đi.

Có một cách dùng Phần D làm nó có giá trị hơn hẳn một danh sách phần mềm, và tôi khuyến nghị nó: *đọc mã nguồn như một phép kiểm tra xem mình đã hiểu lý thuyết đúng chưa*. Cụ thể, có đúng ba tệp đáng đọc thật sự trong toàn bộ hệ sinh thái marker. Tệp `apriltag_pose.c` của [apriltag](#) cho thấy IPPE và hai nghiệm được cài ra sao và nó tính tỉ số sai số tái chiếu như thế nào — tức toàn bộ chương 4 và chương 10 gói trong vài trăm dòng. Phần `objdetect` của [OpenCV](#) cho thấy `ArUco` và `ChArUco` khác nhau ở đúng chỗ nào, và vì sao `ChArUco` có thêm một bước tinh chỉnh mà `ArUco` không có — tức chương 7 và chương 8. Và hàm `solvePnPGeneric` cho thấy cái mà cả chương 3 gọi là “tập nghiệm” trông thế nào khi nó là một mảng thật.

Hai chương của phần này chia theo một ranh giới đơn giản. Chương 17 nói về thứ nằm trong vòng lặp thời gian thực: detector và bộ giải PnP — những thứ chạy ở mỗi khung hình. Chương 18 nói về thứ chạy một lần rồi cho ra một tệp tham số hoặc một con số nghiệm thu: hiệu chuẩn, hand-eye, photogrammetry, visual servoing framework, đánh giá quỹ đạo. Ranh giới ấy cũng gần trùng với ranh giới “thứ bạn phải tự cài” và “thứ bạn nên dùng đồ có sẵn”.

Cảnh báo về thời hạn áp cho cả phần: nội dung chốt tại **tháng 9/2026**. Các tên hàm, tên cờ và cấu trúc module ở đây đúng tại thời điểm ấy và cần rà lại mỗi sáu tháng. Phần lý thuyết ở Phần A tới C thì không có vấn đề này — hình học phối cảnh không đổi phiên bản.

Bảng 1. Bản đồ Phần D: hai chương và ranh giới giữa chúng.

Ch.	Thư mục	Nội dung	Mức
17	17-thu-vien-va-detector	Thứ chạy mỗi khung hình: apriltag (và <code>apriltag_pose.c</code>), OpenCV <code>objdetect/ArUco/ChArUco</code> , các cờ <code>SOLVEPNP_*</code> và <code>solvePnPGeneric</code> , <code>solvePnPRefineLM</code> ; các họ tag khác (<code>STag</code> , <code>ArUco3</code> , <code>CCTag</code> , <code>WhyCon</code> , <code>TopoTag</code> , <code>ChromaTag</code>); detector học sâu; ROS wrapper	●●
18	18-cong-cu-calib-servo	Thứ chạy một lần: hiệu chuẩn (OpenCV , Kalibr , deltille), hand-eye (<code>calibrateHandEye</code> , easy_handeye), BA (Ceres , GTSAM , tagslam), photogrammetry (COLMAP , Metashape), servoing (ViSP), đánh giá (evo)	●●

Chốt tại tháng 9/2026, rà lại mỗi sáu tháng. Chỉ ba thứ đáng đọc mã chứ không chỉ dùng: `apriltag_pose.c` (chương 4 và 10 thành code), [OpenCV](#) `objdetect` (chương 7 và 8), và [ViSP](#) phần IBVS (chương 13). Ba thứ đáng dùng mà không cần đọc: [Kalibr](#), [COLMAP](#), [Ceres](#).

Thư viện và detector (Libraries and Detectors)

PHỤ THUỘC B/07-detection-pipeline (quy trình phát hiện — chương này chỉ ra chỗ nào trong mã hiện thực hoá từng bước); B/08-corner-refinement (tinh chỉnh góc); A/03-pnp-va-uoc-luong-tu-the và A/04-pose-ambiguity-target-phang (PnP và hai nghiệm — solvePnPGeneric là API quan trọng nhất của cả cây).

TÓM TẮT

Chương này là tăng công cụ cho những gì chạy *mỗi khung hình*: detector và bộ giải PnP. Nó không nhằm liệt kê đủ mà nhằm ba việc cụ thể. Thứ nhất, chỉ ra *một* API quan trọng hơn mọi API khác — solvePnPGeneric, vì nó là API duy nhất trả về nhiều nghiệm kèm sai số tái chiếu của từng nghiệm, tức đại lượng cần để đo ambiguity. Thứ hai, chỉ ra ba tệp mã nguồn đáng *đọc* chứ không chỉ dùng, vì đọc chúng là cách kiểm xem mình đã hiểu lý thuyết đúng chưa. Thứ ba, liệt kê các tham số thực sự đáng chỉnh và nói rõ phần còn lại nên để mặc định. Nếu chỉ nhớ một điều: *nếu pipeline của bạn không lấy ra ρ và hiệp phương sai của pose, bạn đang vứt đi hai đại lượng đã được tính sẵn* — và cả cây này dựa vào chúng.

KIẾN THỨC CẦN CÓ TRƯỚC

- **solvePnPGeneric** — hàm OpenCV giải PnP và trả về *mảng* nghiệm cùng *mảng* sai số tái chiếu tương ứng. Khác với solvePnP vốn chỉ trả một nghiệm.
- cờ **SOLVEPNP_*** — tham số chọn thuật toán: ITERATIVE, EPNP, P3P, IPPE, IPPE_SQUARE, SQPNP và vài cái khác. Chúng không tương đương — xem bảng ở mục 3A.
- **objdetect** — module của OpenCV chứa ArUco và ChArUco từ phiên bản 4.7 trở đi; trước đó chúng nằm ở contrib.
- **wrapper ROS** — gói ROS bọc một thư viện gốc để nó nhận và trả dữ liệu qua topic. Tiện, nhưng nó thường *ẩn* các tham số quan trọng và *vứt* các đại lượng chẩn đoán.

CÂU HỎI MỞ ĐẦU

- Q1. Trong toàn bộ các API của OpenCV liên quan tới marker, có đúng một hàm mà thiếu nó thì bạn không thực hiện được B/10. Hàm nào, và đại lượng nào nó cho mà các hàm khác giấu?
- Q2. Bạn gọi solvePnP với cờ mặc định trên bốn góc của một tag vuông. Nếu hai điều không tối ưu đang xảy ra, và cách sửa mỗi điều.
- Q3. Có ba tệp mã nguồn trong cả hệ sinh thái đáng đọc chứ không chỉ dùng. Chúng là gì, và mỗi tệp kiểm được sự hiểu của bạn về chương nào?
- Q4. Wrapper ROS cho detector marker tiện lợi nhưng nó thường làm mất hai thứ mà cây này coi là bắt buộc. Hai thứ nào?
- Q5. Chương này chốt tại tháng 9/2026 và sẽ hết hạn. Phần nào của nó hết hạn nhanh nhất, và phần nào gần như không hết hạn?

1.1 Đặt vấn đề

Phần A tới C không nhắc tới một dòng code cụ thể nào, và đó là có chủ ý: hình học phối cảnh không đổi phiên bản, còn tên hàm thì đổi. Chương này là chỗ trả nợ, và nó mang theo một hạn sử dụng.

Mục đích của chương không phải là một danh mục. Các thư viện đều có tài liệu tốt hơn bất cứ thứ gì tôi viết ở đây, và chép lại chúng là vô ích. Mục đích là ba việc mà tài liệu chính thức *không* làm.

Việc thứ nhất: chỉ ra cái gì quan trọng. Tài liệu API liệt kê mọi hàm với cùng trọng số. Nhưng với bài toán của cây này, có đúng một hàm mà thiếu nó thì cả B/10 không thực hiện được, và có ba tham số đáng chỉnh trong khi vài chục tham số khác nên để mặc định. Biết điều đó tiết kiệm nhiều thời gian hơn biết thêm mười hàm.

Việc thứ hai: nói cái gì bị giấu. Nhiều đại lượng chẩn đoán quan trọng *đã được tính* bên trong thư viện rồi bị vứt đi ở giao diện. Tỉ số ambiguity là ví dụ rõ nhất: IPPE tính cả hai nghiệm dù bạn có dùng hay không, nhưng solvePnP chỉ trả về một. Biết chỗ nào có đại lượng bị giấu là biết chỗ nào đáng sửa vài dòng.

Việc thứ ba, và nó là việc có giá trị lâu dài nhất: chỉ ra mã nào đáng đọc như một bài kiểm tra. Đọc `apriltag_pose.c` rồi nhận ra mình hiểu vì sao nó tính hai nghiệm — đó là một phép kiểm xem A/04 đã được hiểu hay chỉ được đọc. Mục 4 dành cho ba tệp như vậy.

CẠM BẮY

Chương này chốt tại tháng 9/2026 và là chương hết hạn nhanh nhất của cây.

Tên hàm đổi, module được chuyển vị trí (ArUco đã chuyển từ contrib sang objdetect ở OpenCV 4.7), tham số mặc định đổi giữa các phiên bản, và các thư viện mới xuất hiện.

Phần *không* hết hạn: các nguyên tắc ở mục 2 (đại lượng nào cần lấy ra), lý do đằng sau việc chọn cờ ở mục 3, và giá trị của việc đọc mã ở mục 4. Chúng vẫn đúng khi tên hàm đã đổi.

Rà lại mỗi sáu tháng, và khi rà thì kiểm bằng cách *chạy* chứ không bằng cách đọc tài liệu — mục 3B là một ví dụ về việc chạy thật phát hiện ra gì.

1.2 Đại lượng phải lấy ra khỏi pipeline

Trước khi nói về hàm nào, nói về *cái gì* phải có ở đầu ra. Ba đại lượng, và cả ba thường bị vứt.

Một, tỉ số ambiguity ρ . Không có nó thì B/10 bậc 2 không thực hiện được, và bạn không biết khi nào pose của mình không đáng tin. Nó đã được tính sẵn bởi IPPE.

Hai, hiệp phương sai của pose $\mathcal{I}^{-1}$. Không có nó thì B/11 không có trọng số, B/09 mục 7 không có cổng chi-square, và B/13 mục 8 không có tiêu chí “phép đo đáng tin”. Nó đã được tính sẵn bởi bước tinh chỉnh — nó chính là ma trận của phương trình chuẩn (A/03 mục 4B).

Ba, hiệp phương sai của từng góc. Không có nó thì trọng số ở A/03 mục 6 là trọng số đều. Nó đã được tính sẵn bởi bước tinh chỉnh góc — nghịch đảo ma trận cấu trúc (B/08 mục 2).

Ba đại lượng, ba lần “đã được tính sẵn”. Đây là chủ đề của chương: *phần lớn thông tin chẩn đoán mà cây này cần đã tồn tại bên trong thư viện, và việc của bạn là lấy nó ra chứ không phải tính lại.*

Chi phí: với ρ thì đổi một lời gọi hàm; với hai đại lượng còn lại thì vài chục dòng, vì OpenCV không xuất chúng qua API công khai. Đây là một thiếu sót đáng tiếc của API và nó là lý do chính đáng nhất để tự viết bước tinh chỉnh PnP thay vì dùng solvePnPRefineLM.

1.3 OpenCV: các API và một phép chạy thật

A. Chọn cờ SOLVEPNP_* nào

Bảng 1.1. Các cờ SOLVEPNP_* và khi nào dùng cho bài toán marker.

Cờ	Số điểm	Dùng khi
IPPE_SQUARE	đúng 4, tag vuông	Mặc định cho một tag vuông. Trả hai nghiệm kèm reprojection ($A/04$)
IPPE	≥ 4 , đồng phẳng	Target phẳng không vuông, chẳng hạn ChArUco một mặt phẳng
SQPNP	≥ 3 , tổng quát	Mặc định cho constellation không đồng phẳng; tối ưu toàn cục ở mức đại số [1]
EPNP	≥ 4 , tổng quát	Nhiều điểm, cần $O(n)$ [2]
P3P / AP3P	đúng 4	Hạt nhân RANSAC, không dùng làm bộ giải chính
ITERATIVE	≥ 4	Cần khởi tạo tốt; hữu ích với useExtrinsicGuess

Câu trả lời cho Q2: gọi `solvePnP` với cờ mặc định (`ITERATIVE`) trên bốn góc tag vuông có hai điều không tối ưu. Thứ nhất, nó *không* dùng cấu trúc phẳng nên nó không cho bạn hai nghiệm — sửa bằng cách chuyển sang `solvePnPGeneric` với `IPPE_SQUARE`. Thứ hai, `ITERATIVE` không có khởi tạo tốt nên nó có thể hội tụ về nghiệm sai — và nó không báo. Sửa bằng cách khởi tạo bằng lời giải dạng đóng rồi mới tinh chỉnh, đúng như $A/03$ mục 9 khuyến nghị.

B. Một phép chạy thật, và điều nó phát hiện

Tôi kiểm các khẳng định trên bằng cách chạy chứ không bằng cách đọc tài liệu, trên OpenCV 4.13.0 ngày 2026-09-14. Hai điều đáng ghi lại.

Điều thứ nhất, xác nhận: `solvePnPGeneric` với `IPPE_SQUARE` trả về đúng hai nghiệm ở mọi tư thế đã thử, kể cả fronto-parallel, kèm hai giá trị sai số tái chiếu. Tỉ số của chúng là ρ , và $C/16$ mục 3B báo cáo nó biến thiên từ 1,33 tới 66,6 theo góc nghiêng. Khẳng định trung tâm của chương — rằng API này cho bạn đại lượng cần thiết — **đã được kiểm.**

Điều thứ hai, một cái bẫy: với tag ở tư thế fronto-parallel *chính xác*, hai nghiệm có sai số tái chiếu bằng nhau tới sai số số học, nên ρ tính được là ∞ hoặc nan tùy vào việc mẫu số có đúng bằng không hay không. Trong dữ liệu không nhiều, tôi gặp cả hai. Mã của bạn *phải* xử lý trường hợp này — kiểm mẫu số trước khi chia — vì nếu không thì ở đúng tư thế nguy hiểm nhất, chỉ báo an toàn của bạn trả về nan và mọi so sánh với ngưỡng đều thành false.

Đây là một ví dụ nhỏ nhưng đáng nhớ của việc vì sao phải chạy: đọc tài liệu không cho bạn biết điều này, và nó là chế độ hỏng ở đúng chỗ mà chỉ báo cần hoạt động nhất.

C. ArUco và ChArUco

Từ OpenCV 4.7, chúng ở `objdetect` thay vì `contrib`. Lớp chính là `ArucoDetector` và `CharucoDetector`.

Tham số đáng chỉnh, và nó khớp với ba tham số ở $B/07$ mục 5C:

`cornerRefinementMethod` — **bật nó.** Mặc định ở một số phiên bản là `CORNER_REFINE_NONE`, và khi ấy bạn đang lấy góc của đa giác xấp xỉ, kém hơn hẳn ($B/08$). Đặt `CORNER_REFINE_SUBPIX` cho ArUco, và với ChArUco thì bước saddle point đã có sẵn trong `interpolateCornersCharuco`.

`minMarkerPerimeterRate` — ngưỡng kích thước tối thiểu, đặt theo con số đo được của A/02 mục 4 chứ không theo mặc định.

`errorCorrectionRate` — số lỗi bit cho phép sửa. Đặt *thấp* và bù bằng whitelist ID (B/14 mục 7A).

Về từ điển: OpenCV có sẵn nhiều từ điển chuẩn, nhưng với docking thì B/07 mục 5C khuyến nghị từ điển *nhỏ, ít ô bit*. `generateCustomDictionary` tạo được từ điển theo yêu cầu, và nó là một trong số ít chỗ mà đi chệch khỏi mặc định gần như luôn đúng.

1.4 Ba tệp mã đáng đọc

Mục này trả lời Q3, và nó là mục có giá trị lâu dài nhất của chương.

Tệp thứ nhất: `apriltag_pose.c` của [apriltag](#). Vài trăm dòng, và nó là toàn bộ A/04 cộng B/10 bậc 2 thành code. Bạn sẽ thấy: IPPE được cài ra sao, hai nghiệm được tính thế nào, và sai số tái chiếu của từng nghiệm được dùng để chọn ra sao. Đọc nó và nhận ra mình *dự đoán được* nó làm gì tiếp là một phép kiểm tốt xem chương 4 đã được hiểu hay chỉ được đọc.

Tệp thứ hai: phần `quad fitting` của [apriltag](#) (thường ở `apriltag_quad_thresh.c`). Đây là B/07 mục 2 thành code, và chỗ đáng chú ý nhất là bước khớp đường có trọng số gradient — cải tiến chính của AprilTag 2 (B/07 mục 2). Đọc nó để thấy vì sao độ chính xác dưới pixel là chuyện có thể.

Tệp thứ ba: phần IBVS của [ViSP](#). Đây là B/13 mục 2 thành code: ma trận tương tác được dựng thế nào, và luật $\mathbf{v} = -\lambda \mathbf{L}^+ \mathbf{e}$ được cài ra sao. Đáng đọc nếu bạn cân nhắc IBVS cho pha cuối.

(Ghi chú trung thực: mô tả về nội dung ba tệp này là từ trí nhớ về cấu trúc của chúng; tôi chưa mở lại mã trong lần soạn này. Tên tệp cụ thể có thể đã đổi. Điều tôi tin chắc là chức năng tồn tại trong các thư viện ấy, không phải tên tệp chính xác.)

Ngược lại, ba thứ đáng dùng mà không cần đọc: [Kalibr](#), [COLMAP](#), [Ceres](#). Chúng là công cụ trưởng thành, tài liệu tốt, và việc đọc mã của chúng không trả lại tương xứng với thời gian bỏ ra.

1.5 Các họ tag khác và detector học sâu

Ngắn, vì B/07 mục 4 đã bàn về mặt kỹ thuật; mục này chỉ nói về mặt công cụ.

[STag](#) [3], [TopoTag](#) [4], [ChromaTag](#) [5]: các họ tag với thiết kế khác. Mã nguồn có, chất lượng khác nhau. Đáng thử nếu bạn có một yêu cầu cụ thể mà AprilTag và ArUco không đáp ứng — chẳng hạn STag khi cần ổn định pose tốt hơn ở tư thế xấu.

[CCTag](#) [6], [WhyCon](#) [7]: marker tròn. Nhớ cảnh báo về bias tâm ellipse (A/02 mục 6) — nếu dùng, phải kiểm xem cài đặt có hiệu chỉnh conic không, và nếu không thì bạn có một dòng hệ thống phụ thuộc tư thế trong ngân sách.

[DeepTag](#) [8], [DeepArUco](#) [9]: detector học sâu. Đáng cân nhắc khi điều kiện quan sát khắc nghiệt và không cải thiện được bằng chiếu sáng (B/14). Với docking — nơi điều kiện kiểm soát được — lợi ích ít rõ, và B/07 mục 4 nêu lý do thận trọng: bền hơn không có nghĩa là chính xác hơn về định vị góc.

[apriltag-generation](#): tạo họ tag riêng với số ô và khoảng cách Hamming tùy chọn. Công cụ để thực hiện đánh đổi ở B/07 mục 5C, và nó đáng dùng hơn vẻ ngoài vì lựa chọn mặc định (dùng họ phổ biến nhất) thường không tối ưu cho docking.

1.6 Wrapper ROS

Mục này trả lời Q4, và nó là một cảnh báo.

`apriltag_ros`, `aruco_ros`, `fiducials`: các gói ROS bọc thư viện gốc. Chúng tiện — nhận ảnh qua topic, trả pose qua TF — và với một hệ thống ROS thì chúng là điểm xuất phát hợp lý.

Nhưng chúng thường làm mất hai thứ mà cây này coi là bắt buộc.

Thứ nhất, ρ . Giao diện ROS thường trả về một pose duy nhất, nên tỉ số ambiguity bị vớt ở tầng wrapper. Hệ quả: bạn không thực hiện được B/10 bậc 2, và bạn không biết khi nào pose không đáng tin.

Thứ hai, hiệp phương sai trung thực. Một số wrapper trả về một trường hiệp phương sai, và nó thường là một hằng số cầu hình chữ không phải $\mathcal{I}^{-1}$ tính từ dữ liệu. Một hiệp phương sai hằng số tệ hơn không có hiệp phương sai, vì nó trông như thông tin thật và mọi tầng trên sẽ tin nó.

Hai mất mát ấy là lý do tôi khuyên: *dùng wrapper để bắt đầu, nhưng sửa nó hoặc thay nó trước khi nghiêm túc.* Việc sửa không lớn — lấy ρ ra và thêm nó vào message — và nó là điều kiện để phần lớn Phần B áp dụng được.

Một điều đáng nêu về `tagslam` [10]: nó khác các wrapper trên ở chỗ nó giải bài toán hợp nhất trên nhiều tag bằng factor graph, tức nó làm đúng việc mà B/09 mục 2 khuyến nghị. Nếu bạn cần map nhiều tag trong môi trường thì nó là điểm xuất phát tốt. Mức tin C — tiền ẩn, nhưng có mã nguồn công khai và được dùng rộng rãi.

1.7 Giới hạn

Chương này không có benchmark. B/07 mục 6 giải thích: tôi không có dữ liệu so sánh được giữa các detector, và trích số từ các bài báo khác nhau ra khỏi ngữ cảnh thì chúng không so sánh được. Nếu cần xếp hạng cho hệ của bạn, mini project của B/07 là cách đo.

Các khẳng định về mã nguồn là từ trí nhớ. Đã ghi ở mục 4. Tên tệp có thể đã đổi; chức năng thì tôi tin là còn.

Chỉ các khẳng định về OpenCV API đã được chạy kiểm, trên đúng một phiên bản (4.13.0) và một hệ điều hành. Hành vi có thể khác ở phiên bản khác — và đặc biệt, vị trí module `ArUco` đã đổi một lần rồi.

Chương chốt tháng 9/2026. Xem cảnh báo ở mục 1.

1.8 Thực hành

Bốn việc.

Một, chuyển sang `solvePnPGeneric` và lấy ρ ra — việc có tỉ lệ lợi ích trên công sức cao nhất trong chương, và nó là điều kiện cho B/10.

Hai, xử lý trường hợp ρ là nan hoặc vô hạn — mục 3B; nó xảy ra ở đúng tư thế nguy hiểm nhất.

Ba, bật tính chỉnh góc nếu dùng `ArUco`, và kiểm giá trị mặc định của phiên bản bạn đang dùng thay vì giả định.

Bốn, nếu dùng wrapper ROS, sửa nó để xuất ρ trước khi xây dựng gì phía trên.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
●●● API quan trọng nhất của cả cây	<code>solvePnPGeneric</code> — API duy nhất trả về <i>nhiều nghiệm kèm sai số tái chiếu của từng nghiệm</i> , tức ρ . Thiếu nó thì B/10 bậc 2 không thực hiện được.
●●● Ba đại lượng phải lấy ra	ρ (cho B/10); hiệp phương sai pose $\mathcal{I}^{-1}$ (cho B/11, cổng chi-square, tiêu chí hội tụ); hiệp phương sai từng góc (cho trọng số ở A/03 mục 6). Cả ba đã được tính sẵn — việc của bạn là lấy ra, không phải tính lại.
●●● Chọn cờ nào	IPPE_SQUARE cho một tag vuông (cho hai nghiệm); SQPNP cho constellation không đồng phẳng; EPNP khi nhiều điểm; P3P chỉ làm hạt nhân RANSAC.
●●● Hai vấn đề của <code>solvePnP</code> mặc định	Cờ ITERATIVE không dùng cấu trúc phẳng nên không cho hai nghiệm; và nó không có khởi tạo tốt nên có thể hội tụ về nghiệm sai <i>mà không báo</i> .
●●● Cái bẫy ρ tại fronto-parallel	Ở tư thế fronto-parallel chính xác, hai sai số tái chiếu bằng nhau tới sai số số học nên ρ ra nan hoặc ∞ . Phải kiểm mẫu số trước khi chia — nếu không thì ở đúng tư thế nguy hiểm nhất, chỉ báo an toàn trả nan và mọi so sánh ngưỡng thành false. Phát hiện bằng cách CHẠY, không bằng đọc tài liệu.
●● Ba tham số ArUco đáng chỉnh	<code>cornerRefinementMethod</code> (bật — mặc định có thể là NONE); <code>minMarkerPerimeterRate</code> (theo ngưỡng đo được của A/02 mục 4); <code>errorCorrectionRate</code> (thấp, bù bằng whitelist).
●●● Ba tệp mã đáng đọc	<code>apriltag_pose.c</code> (A/04 + B/10 thành code); phần quad fitting của <code>apriltag</code> (B/07 mục 2, khớp đường có trọng số gradient); phần IBVS của <code>ViSP</code> (B/13 mục 2).
●● Ba thứ đáng dùng mà không cần đọc	<code>Kalibr</code> , <code>COLMAP</code> , <code>Ceres</code> — công cụ trưởng thành, tài liệu tốt, đọc mã không trả lại tương xứng.
●●● Hai thứ wrapper ROS làm mất	ρ (giao diện trả một pose duy nhất); và hiệp phương sai <i>trung thực</i> (thường là hằng số cấu hình chứ không phải $\mathcal{I}^{-1}$). Hiệp phương sai hằng số <i>tệ hơn</i> không có, vì nó trông như thông tin thật.
●● Vì sao chương này không có benchmark	Không có dữ liệu so sánh được giữa các detector; trích số từ các bài báo khác nhau ra khỏi ngữ cảnh thì chúng không so được (B/07 mục 6). Cần xếp hạng thì đo trên hệ của mình.
●● Phần nào của chương hết hạn	Tên hàm, vị trí module, giá trị mặc định — hết hạn nhanh. <i>Nguyên tắc</i> (đại lượng nào cần lấy ra, lý do chọn cờ, giá trị của việc đọc mã) — gần như không hết hạn.
● Từ điển tùy chỉnh	<code>generateCustomDictionary</code> — một trong số ít chỗ mà đi lệch khỏi mặc định gần như luôn đúng cho docking, vì bạn chỉ cần vài chục ID (B/07 mục 5C).

●●● phải thuộc ●● nên nắm ● biết là có

Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
OpenCV <code>solvePnPGeneric</code>	Nhiều nghiệm kèm reprojection từng nghiệm	Đã kiểm bằng cách chạy, 4.13.0, 2026-09-14
<code>apriltag</code> (AprilRobotics)	Bản C gốc; <code>apriltag_pose.c</code> và phần quad fitting	Hai trong ba tệp đáng đọc
OpenCV <code>objdetect</code>	ArUco và ChArUco từ 4.7; <code>ArucoDetector</code> , <code>CharucoDetector</code>	Nhớ kiểm mặc định của <code>cornerRefinementMethod</code>
<code>ViSP</code> [11]	IBVS, PBVS, ma trận tương tác, tracker marker	Tệp thứ ba đáng đọc; cho B/13

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
tagslam [10]	Hợp nhất nhiều tag bằng factor graph — đúng việc B/09 mục 2 khuyến nghị	Khác các wrapper ROS khác ở điểm này; mức tin C

MINI PROJECT · Sửa pipeline để xuất ba đại lượng chẩn đoán · 1 ngày

Mục tiêu	biến pipeline của bạn từ một hộp đen trả về một pose thành một hệ đo lường trả về pose kèm bằng chứng về độ tin cậy của nó.
Dữ liệu	chính pipeline hiện tại của bạn.
Các bước	chuyển lời gọi PnP sang solvePnPGeneric với cờ đúng theo bảng ở mục 3A; lấy mảng reprojection ra và tính ρ , có xử lý trường hợp mẫu số bằng không; tự viết bước tinh chỉnh LM (hoặc sửa solvePnPRefineLM) để lấy ra ma trận $\mathcal{I} = \sum J^T \Sigma^{-1} J$ và nghịch đảo nó; nếu dùng ArUco, lấy ma trận cấu trúc từ bước cornerSubPix để có hiệp phương sai từng góc; cuối cùng đối giao diện trả về từ một pose thành cấu trúc ba giá trị của B/10 mục 3C.
Xong khi	pipeline trả về pose, hiệp phương sai, ρ , và một trạng thái ba giá trị; và log của bạn ghi cả ba ở mỗi khung. Kiểm tra chéo bắt buộc: chạy trên một chuỗi có đoạn fronto-parallel và xác nhận rằng ρ không trả về nan ở đó — nếu có thì bạn chưa xử lý trường hợp ở mục 3B, và bạn vừa mất chỉ báo ở đúng chỗ cần nó nhất.
Nối với	B/10 và B/13 mục 8 tiêu thụ trực tiếp ba đại lượng này.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. solvePnPGeneric. Nó là API duy nhất trả về *mảng* nghiệm cùng *mảng* sai số tái chiếu tương ứng; mọi hàm khác — kể cả solvePnP với cùng cờ IPPE — chỉ trả về một nghiệm và vứt phần còn lại. Đại lượng nó cho mà các hàm khác giấu là tỉ số $\rho = e_2/e_1$, tức chỉ báo duy nhất cho biết hai nghiệm có phân biệt được hay không. Thiếu nó thì bậc 2 của B/10 — đo và từ chối khi không tin được — không thực hiện được, và hệ của bạn sẽ chọn nghiệm theo nhiều ở những tư thế mà hai nghiệm ngang nhau, không có cảnh báo nào. Điều đáng nói là IPPE *đã tính* cả hai nghiệm dù bạn có dùng hay không, nên chi phí để lấy ρ ra bằng không — chỉ là đổi một lời gọi hàm.
- A2. Thứ nhất, cờ mặc định ITERATIVE không khai thác cấu trúc phẳng của tag vuông, nên nó không cho bạn hai nghiệm và bạn mất ρ . Sửa: dùng solvePnPGeneric với SOLVEPNP_IPPE_SQUARE. Thứ hai, ITERATIVE là một bộ giải lặp cần điểm khởi đầu, và nếu không truyền useExtrinsicGuess thì nó tự khởi tạo — nên nó có thể hội tụ về nghiệm sai trong hai nghiệm của target phẳng, và nó *không báo* là đã làm thế. Sửa: khởi tạo bằng một lời giải dạng đóng (IPPE cho target phẳng) rồi mới tinh chỉnh, đúng như A/03 mục 9 khuyến nghị. Cả hai lỗi này đều thầm lặng — hàm trả về một pose trông hợp lý trong cả hai trường hợp.
- A3. Một: apriltag_pose.c của [apriltag](#) — nó là A/04 (hai nghiệm của target phẳng) cộng B/10 bậc 2 (dùng tỉ số reprojection để chọn) gói trong vài trăm dòng; đọc nó và thấy mình *dự đoán được* bước tiếp theo là phép kiểm xem chương 4 đã được hiểu hay chỉ được đọc. Hai: phần quad fitting của [apriltag](#) — nó là B/07 mục 2, và chỗ đáng chú ý nhất là bước khớp đường có trọng số gradient, tức cơ chế làm cho độ chính xác dưới pixel khả thi. Ba: phần IBVS của [ViSP](#) — nó là B/13 mục 2, cho thấy ma trận tương tác được dựng thế nào và luật điều khiển được cài ra sao. Ngược lại, [Kalibr](#), [COLMAP](#) và [Ceres](#) thì nên dùng mà không cần đọc: chúng trưởng thành, tài liệu tốt, và đọc mã không trả lại tương xứng thời gian.
- A4. Thứ nhất, tỉ số ambiguity ρ : giao diện ROS thường trả về một pose duy nhất qua TF hoặc một message pose, nên ρ bị vứt ở tầng wrapper — và không có nó thì bạn không biết khi nào pose của mình không đáng tin. Thứ hai, hiệp phương sai *trung thực*: một số wrapper có trường hiệp phương sai trong message, nhưng nó thường được điền bằng một hằng số lấy từ tệp cấu hình chứ không phải $\mathcal{I}^{-1}$ tính từ dữ liệu của khung hiện tại. Mất mát thứ hai tệ hơn mất mát thứ nhất, vì một hiệp phương sai hằng số *tệ hơn không có hiệp phương sai*: nó trông như thông tin thật, nên mọi tầng phía trên — bộ lọc, cổng chi-square, tiêu chí hội tụ — sẽ tin nó và cân các

phép đo sai. Khuyến nghị: dùng wrapper để bắt đầu, nhưng sửa nó để xuất ρ trước khi xây dựng gì phía trên; việc sửa nhỏ và nó là điều kiện để phần lớn Phần B áp dụng được.

- A5. Hết hạn nhanh nhất: tên hàm, vị trí module (ArUco đã chuyển từ `contrib` sang `objdetect` ở 4.7 — một lần đổi đã xảy ra rồi), giá trị mặc định của tham số, và danh sách các thư viện mới. Gần như không hết hạn: các *nguyên tắc* — rằng có ba đại lượng chẩn đoán cần lấy ra và cả ba đã được tính sẵn; rằng phải chọn cờ theo cấu trúc hình học của target chứ không theo mặc định; rằng đọc mã là một phép kiểm sự hiểu chứ không chỉ là cách học API; và rằng wrapper tiện lợi thường vứt đi thông tin chẩn đoán. Bốn nguyên tắc ấy vẫn đúng khi mọi tên hàm trong chương đã đổi. Cách rà lại đúng: kiểm bằng cách *chạy* chứ không bằng đọc tài liệu — mục 3B là ví dụ về việc chạy thật phát hiện ra một cái bẫy mà tài liệu không nói.

NÓI LẠI THẬT ĐƠN GIẢN

Phần lớn tài liệu này nói về nguyên lý, và nguyên lý thì không đổi. Chương này nói về tên hàm, và tên hàm thì đổi — nên nó có hạn sử dụng, ghi rõ là tháng 9 năm 2026.

Điều đáng nhớ nhất của chương không phải một tên hàm mà là một quan sát: *phần lớn thông tin bạn cần đã được máy tính sẵn rồi, chỉ là nó bị vứt đi ở cửa ra.*

Ví dụ rõ nhất là chuyện hai câu trả lời của cái tag phẳng. Thư viện *luôn* tính cả hai — nó không có cách nào tính một mà không tính cái kia — nhưng cái hàm thông dụng nhất chỉ trả về một và ném cái còn lại đi. Đổi sang một hàm khác, và bạn có thêm con số cho biết hai câu trả lời có ngang nhau hay không. Đổi một dòng, được một chỉ báo an toàn.

Tương tự với chuyện “máy tin vào kết quả của mình tới mức nào”. Con số đó cũng được tính sẵn trong lúc giải, rồi bị vứt. Lấy nó ra tốn vài chục dòng, và không có nó thì một nửa những việc ở phần trước của tài liệu không làm được.

Có một cái bẫy nhỏ mà tôi chỉ phát hiện được nhờ chạy thử chứ không nhờ đọc tài liệu, và nó đáng kể lại vì nó rất đặc trưng. Cái tỉ số so sánh hai câu trả lời được tính bằng một phép chia. Ở đúng cái tư thế nguy hiểm nhất — tag nhìn thẳng vào camera — hai câu trả lời khớp ảnh ngang nhau hoàn hảo, nên mẫu số bằng không, nên phép chia cho ra một giá trị vô nghĩa. Và một giá trị vô nghĩa thì khi đem so với ngưỡng sẽ luôn ra “không đạt ngưỡng” theo cách im lặng. Nghĩa là: nếu không cẩn thận, cái chỉ báo an toàn của bạn sẽ tắt ngúm đúng vào lúc bạn cần nó nhất.

Cuối cùng, một lời khuyên về việc đọc mã người khác. Có ba tệp đáng đọc trong cả hệ sinh thái này, và lý do đọc chúng không phải để học cách dùng thư viện — tài liệu làm việc đó tốt hơn. Lý do là để *tự kiểm tra mình*: nếu bạn đọc một tệp và thấy mình đoán trước được dòng tiếp theo làm gì, thì bạn đã hiểu cái chương lý thuyết tương ứng. Nếu bạn đọc mà thấy lạ, thì chương ấy mới chỉ được đọc chứ chưa được hiểu.

Chỗ không rút gọn được: tên hàm, tên module và giá trị mặc định sẽ đổi, và không có cách nào viết một chương như thế này cho nó bền. Cách duy nhất là mỗi lần quay lại thì *chạy thử* chứ đừng tin cái đã viết — kể cả cái tôi vừa viết.

Một câu: *Thông tin bạn cần đã được tính sẵn rồi; việc của bạn là đừng để nó bị vứt ở cửa ra.*

1.9 Điều gì chứng minh cách hiểu này sai

Chương công cụ khác chương lý thuyết ở chỗ phần lớn khẳng định của nó kiểm được bằng cách chạy, và một phần đã được kiểm — nên mục này nói về phần còn lại.

Khẳng định trung tâm — `solvePnPGeneric` với `IPPE_SQUARE` trả về hai nghiệm kèm sai số tái chiếu, và tỉ số của chúng là một chỉ báo dùng được — **đã được kiểm bằng cách chạy** (mục 3B), trên OpenCV 4.13.0. Nó bị bác bỏ nếu một phiên bản khác trả về một nghiệm, hoặc trả về hai nghiệm mà sai số tái chiếu không phân biệt được chúng ở góc nghiêng lớn. Cách kiểm lại khi nâng cấp: chạy phần 2 của script ở C/16 và xem cột ρ có còn tăng theo góc nghiêng không.

Khẳng định thứ hai — *ba đại lượng chẩn đoán đã được tính sẵn và chỉ cần lấy ra* — bị bác bỏ nếu hoá ra một trong ba không tồn tại ở dạng có thể lấy được. Với ρ thì tôi đã kiểm. Với hiệp phương sai

pose và hiệp phương sai góc thì tôi *chưa* — tôi dẫn ra từ cấu trúc của thuật toán (A/03 mục 4B, B/08 mục 2) chứ không từ việc mở mã OpenCV. Nếu hoá ra chúng bị huỷ trước khi hàm trả về thì lời khuyên “lấy ra” phải đổi thành “tự cài bước tinh chỉnh”, và chi phí của mini project tăng đáng kể.

Khẳng định thứ ba — *ba tệp mã đáng đọc* — là phán đoán chủ quan của tôi về giá trị sư phạm, và nó không bị bác bỏ bằng dữ liệu. Điều *có thể* sai là mô tả về nội dung của chúng, vốn từ trí nhớ và chưa mở lại mã (mục 4).

Cuối cùng, khẳng định về wrapper ROS ở mục 6 dựa trên hiểu biết chung về các gói ấy, *không* phải trên việc đọc mã của chúng trong lần soạn này. Nếu một wrapper cụ thể có xuất ρ thì cảnh báo ở mục 6 không áp dụng cho nó — và đó sẽ là tin tốt đáng ghi lại.

1.10 Kết nối

Chương này là tầng công cụ cho những gì chạy mỗi khung.

Nổi lên trên. B/07-detection-pipeline mô tả quy trình mà mục 3C và 4 chỉ ra chỗ hiện thực hoá. B/08-corner-refinement mô tả các phương pháp mà tham số cornerRefinementMethod chọn giữa. A/03-pnp-va-uoc-luong-tu-the và A/04-pose-ambiguity-target-phang là hai chương mà solvePnPGeneric phục vụ trực tiếp.

Nổi ngang. D/18-cong-cu-calib-servo-va-do-dac là chương anh em: nó bàn những gì chạy *một lần* thay vì mỗi khung, và ranh giới ấy gần trùng với ranh giới “thứ bạn phải tự cài” và “thứ nên dùng đồ có sẵn”.

Nổi xuống. B/10-xu-ly-ambiguity và B/13-docking-control mục 8 tiêu thụ trực tiếp ba đại lượng ở mục 2, nên mini project của chương này là điều kiện tiên quyết cho cả hai.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Nêu ba đại lượng phải lấy ra khỏi pipeline, chương nào tiêu thụ mỗi cái, và điều chung của cả ba.
2. Vì sao gọi solvePnP với cờ mặc định trên tag vuông là không tối ưu theo hai cách? Nêu cách sửa mỗi cách.
3. Mô tả cái bẫy ρ tại fronto-parallel, và nói vì sao nó nguy hiểm hơn một lỗi thông thường.
4. Ba tệp mã nào đáng đọc, và mỗi tệp kiểm được sự hiểu của bạn về chương nào?
5. Wrapper ROS làm mất hai thứ. Nêu cả hai, và nói vì sao mất mát thứ hai tệ hơn mất mát thứ nhất.
6. Phần nào của chương này hết hạn nhanh nhất và phần nào không? Nêu cách rà lại đúng.
7. Vì sao chương này cố ý không có bảng benchmark so sánh các detector?

Nguồn

- [1] G. Terzakis and M. Lourakis, “A Consistently Fast and Globally Optimal Solution to the Perspective-n-Point Problem,” in *European Conference on Computer Vision (ECCV)*, mức tin B; SQPnP, có trong OpenCV, 2020.
- [2] V. Lepetit, F. Moreno-Noguer, and P. Fua, “EPnP: An Accurate $O(n)$ Solution to the PnP Problem,” *International Journal of Computer Vision*, vol. 81, no. 2, pp. 155–166, 2009, mức tin A.
- [3] B. Benligray, C. Topal, and C. Akinlar, “STag: A Stable Fiducial Marker System,” in *Image and Vision Computing*, mức tin B; dùng đường tròn ngoại tiếp để ổn định pose, vol. 89, 2019, pp. 158–169.
- [4] G. Yu, Y. Hu, and J. Dai, “TopoTag: A Robust and Scalable Topological Fiducial Marker System,” in *IEEE Transactions on Visualization and Computer Graphics*, mức tin B, 2021.
- [5] J. DeGol, T. Bretl, and D. Hoiem, “ChromaTag: A Colored Marker and Fast Detection Algorithm,” in *IEEE International Conference on Computer Vision (ICCV)*, mức tin B, 2017.
- [6] L. Calvet, P. Gurdjos, C. Griwodz, et al., “Detection and Accurate Localization of Circular Fiducials under Highly Challenging Conditions,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; CCTag, vòng tròn đồng tâm định vị chính xác, 2016.
- [7] T. Krajník, M. Nitsche, J. Faigl, et al., “A Practical Multirobot Localization System,” *Journal of Intelligent & Robotic Systems*, vol. 76, pp. 539–562, 2014, mức tin B; WhyCon, marker tròn đồng tâm.
- [8] Z. Zhang, Y. Hu, G. Yu, et al., “DeepTag: A General Framework for Fiducial Marker Design and Detection,” in *IEEE Transactions on Pattern Analysis and Machine Intelligence*, mức tin B, 2023.
- [9] R. Berral-Soler, F. J. Madrid-Cuevas, R. Muñoz-Salinas, et al., “DeepArUco: Marker Detection and Classification in Challenging Lighting Conditions,” *IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2023, mức tin C; tiền ấn/-workshop, dùng làm gợi ý.
- [10] B. Pfrommer and K. Daniilidis, “TagSLAM: Robust SLAM with Fiducial Markers,” in *arXiv preprint arXiv:1910.00679*, mức tin C; tiền ấn nhưng có mã nguồn công khai và được dùng rộng rãi — dùng làm gợi ý về kiến trúc, không làm bằng chứng số, 2019.
- [11] É. Marchand, F. Spindler, and F. Chaumette, “ViSP for Visual Servoing: A Generic Software Platform with a Wide Class of Robot Control Skills,” *IEEE Robotics & Automation Magazine*, vol. 12, no. 4, pp. 40–52, 2005, mức tin A.
- [12] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; COLMAP, 2016.
- [13] Z. Zhang, “A Flexible New Technique for Camera Calibration,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 11, pp. 1330–1334, 2000, mức tin A; phương pháp hiệu chuẩn từ target phẳng mà mọi hộp công cụ đang dùng.
- [14] P. Furgale, J. Rehder, and R. Siegwart, “Unified Temporal and Spatial Calibration for Multi-Sensor Systems,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; nền của Kalibr, 2013.

- [15] H. Ha, M. Perdoch, H. Alismail, et al., “Deltille Grids for Geometric Camera Calibration,” in *IEEE International Conference on Computer Vision (ICCV)*, mức tin A; lưới tam giác và tính chỉnh saddle point bậc hai, chính xác hơn checkerboard, 2017.
- [16] S. Agarwal, K. Mierle, et al., “Ceres Solver,” <http://ceres-solver.org>, 2012, mức tin A; phần mềm, không phải bài báo.
- [17] F. Dellaert and M. Kaess, “Factor Graphs for Robot Perception,” *Foundations and Trends in Robotics*, vol. 6, no. 1-2, pp. 1–139, 2017, mức tin A.
- [18] R. Y. Tsai and R. K. Lenz, “A New Technique for Fully Autonomous and Efficient 3D Robotics Hand/Eye Calibration,” *IEEE Transactions on Robotics and Automation*, vol. 5, no. 3, pp. 345–358, 1989, mức tin A; lời giải kinh điển cho $AX=XB$.
- [19] F. C. Park and B. J. Martin, “Robot Sensor Calibration: Solving $AX=XB$ on the Euclidean Group,” *IEEE Transactions on Robotics and Automation*, vol. 10, no. 5, pp. 717–721, 1994, mức tin A.
- [20] K. Daniilidis, “Hand-Eye Calibration Using Dual Quaternions,” *The International Journal of Robotics Research*, vol. 18, no. 3, pp. 286–298, 1999, mức tin A; giải đồng thời xoay và tịnh tiến.
- [21] M. Grupp, “evo: Python Package for the Evaluation of Odometry and SLAM,” <https://github.com/MichaelGrupp/evo>, 2017, mức tin B; phần mềm.
- [22] Z. Zhang and D. Scaramuzza, “A Tutorial on Quantitative Trajectory Evaluation for Visual(-Inertial) Odometry,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; căn chỉnh quỹ đạo trước khi tính sai số, và vì sao chọn sai phép căn chỉnh làm con số vô nghĩa, 2018.

Công cụ hiệu chuẩn, servoing và đo đạc (One-Time Tools)

PHỤ THUỘC B/12-calibration-cho-docking (ba phép hiệu chuẩn và thứ tự bắt buộc của chúng — chương này nói dùng công cụ nào cho từng bước); B/09-multi-marker-constellation mục 5 (đo constellation bằng BA); B/13-docking-control (servoing); C/16-danh-gia-va-nghiem-thu (đánh giá).

TÓM TẮT

Chương này là tăng công cụ cho những gì chạy *một lần* rồi cho ra một tệp tham số hoặc một con số nghiệm thu — khác với D/17 vốn nói về những gì chạy mỗi khung hình. Ranh giới ấy gần trùng với một ranh giới khác đáng chú ý: *thứ bạn nên dùng đồ có sẵn* so với *thứ bạn phải tự cài*. Bốn nhóm công cụ: hiệu chuẩn intrinsic, đo constellation bằng photogrammetry, hand-eye, và đánh giá. Với ba nhóm đầu, lời khuyên đơn giản là dùng công cụ trưởng thành và đừng tự viết. Với nhóm thứ tư thì ngược lại, và lý do đáng nói: các công cụ đánh giá có sẵn được thiết kế cho bài toán quỹ đạo SLAM, còn bài toán docking cần đo *độ lặp lại tại một điểm* — một đại lượng khác, và tự viết vài chục dòng thì đúng hơn là ép một công cụ không hợp. Nếu chỉ nhớ một điều: *ba bước hiệu chuẩn phải chạy đúng thứ tự, và mỗi bước phải kiểm hiệp phương sai của kết quả trước khi dùng nó cho bước sau*.

KIẾN THỨC CẦN CÓ TRƯỚC

- **photogrammetry** — đo hình học ba chiều từ nhiều ảnh. Với cây này, nó là công cụ thực hiện B/09 mục 5: đo lại vị trí thật của các tag thay vì tin bản vẽ.
- **structure from motion (SfM)** — khôi phục đồng thời cấu trúc 3D và tư thế camera từ một tập ảnh không có thông tin tư thế. **COLMAP** là cài đặt phổ biến nhất [12].
- **gauge fixing** — việc cố định các bậc tự do mà dữ liệu không xác định được — với BA từ ảnh đơn thuần thì đó là vị trí, hướng, và *tỉ lệ* của toàn bộ cấu trúc.
- **ATE / RPE** — *absolute trajectory error* và *relative pose error*: hai chỉ số chuẩn cho đánh giá quỹ đạo SLAM. Chúng *không* phải chỉ số đúng cho docking — mục 5 giải thích.

CÂU HỎI MỞ ĐẦU

- Q1. Ba bước hiệu chuẩn của B/12 cần ba công cụ khác nhau. Nêu công cụ cho từng bước, và nói bước nào bạn có thể phải tự viết.
- Q2. Bạn dùng **COLMAP** để đo constellation. Nó trả về một cấu trúc 3D đẹp. Vì sao bạn *vẫn chưa* có thứ cần dùng, và còn thiếu một thông tin gì?
- Q3. Các công cụ đánh giá quỹ đạo như **evo** rất trưởng thành. Vì sao chúng lại *không* phải công cụ đúng cho nghiệm thu docking?
- Q4. Sau khi chạy BA đo constellation, có một đại lượng bạn phải nhìn trước khi dùng kết quả — và bỏ qua nó là tự lừa mình. Đại lượng nào, và tự lừa mình thế nào?
- Q5. Trong bốn nhóm công cụ của chương, ba nhóm nên dùng đồ có sẵn và một nhóm nên tự viết. Nhóm nào, và nguyên tắc chung rút ra là gì?

2.1 Đặt vấn đề

D/17 nói về những gì chạy mỗi khung hình. Chương này nói về những gì chạy một lần: hiệu chuẩn, đo đạc, và đánh giá.

Ranh giới ấy quan trọng hơn vẻ ngoài, vì nó gần trùng với một ranh giới khác về mặt kỹ thuật phần mềm. Thứ chạy mỗi khung phải nhanh, phải chạy trên máy của robot, và phải tích hợp chặt với pipeline — nên bạn thường phải tự cài hoặc ít nhất tự sửa. Thứ chạy một lần thì chạy trên máy tính bàn, không có ràng buộc thời gian thực, và nó cho ra một tệp — nên dùng công cụ có sẵn gần như luôn đúng.

“Gần như” vì có một ngoại lệ, và nó là nội dung đáng nói nhất của chương. Công cụ đánh giá có sẵn được thiết kế cho bài toán *quỹ đạo SLAM*: chúng so hai quỹ đạo dài, có các phép căn chỉnh tinh vi, và chúng báo cáo ATE và RPE. Bài toán docking không phải bài toán ấy — nó cần đo *độ lặp lại của một điểm cuối* qua nhiều lần thử. Ép một công cụ quỹ đạo làm việc ấy thì được, nhưng nó phức tạp hơn và dễ sai hơn so với vài chục dòng tự viết. Mục 5 bàn kỹ.

Có một lý do thứ hai để chương này tồn tại, và nó thuộc về quy trình chứ không về công cụ: *thứ tự*. B/12 mục 2 chỉ ra rằng ba phép hiệu chuẩn phải chạy theo đúng thứ tự vì mỗi bước dùng kết quả bước trước làm hằng số. Chương này bổ sung phần thực hành: mỗi bước dùng công cụ nào, và — quan trọng hơn — mỗi bước phải kiểm gì trước khi chuyển sang bước sau. Bỏ qua bước kiểm ấy là cách phổ biến nhất để có một chuỗi hiệu chuẩn trông hoàn chỉnh mà sai.

2.2 Hiệu chuẩn intrinsic

Bước một của B/12 mục 2.

`OpenCV` `calibrateCamera` — đủ dùng cho phần lớn trường hợp, có sẵn, và nó cài phương pháp Zhang [13]. Nhược điểm: nó không cho bạn bản đồ residual theo vị trí ảnh, mà đó là phép kiểm quan trọng nhất (A/01 mục 6). Bạn phải tự vẽ — vài chục dòng, và nó đáng.

`Kalibr` [14] — mạnh hơn, hỗ trợ nhiều mô hình méo, và nó có báo cáo residual theo vị trí. Nó cũng xử lý được nhiều camera và camera-IMU. Giá phải trả: nặng hơn để cài đặt, và nó được thiết kế quanh quy trình ROS bag.

`deltille` [15] — không phải công cụ hiệu chuẩn hoàn chỉnh mà là một detector target chất lượng cao. Dùng nó khi cần intrinsic chính xác nhất và khi target lưới tam giác chấp nhận được.

Về target: B/12 mục 3C khuyến nghị ChArUco hoặc deltille, và nhấn mạnh rằng target phải *phẳng thật* — in trên giấy dán bìa cứng không đủ. Với hiệu chuẩn chất lượng cao, target in trên tấm nhôm hoặc kính là đầu tư đáng.

Phải kiểm trước khi chuyển sang bước hai: bản đồ residual không có cấu trúc (A/01 mục 6), và c_x, c_y ổn định qua các tập con ảnh (B/12 mục 3B). Nếu điểm chính nhảy hàng chục pixel giữa các tập con thì bạn chưa có intrinsic dùng được, và chạy bước hai với nó là lãng phí.

2.3 Đo constellation bằng photogrammetry

Bước hai, và mục này trả lời Q2 và Q4.

A. Ba lựa chọn

Tự viết BA bằng *Ceres* [16] hoặc *GTSAM* [17]. Đây là cách cho kết quả tốt nhất, vì bạn tận dụng được cấu trúc mà một công cụ tổng quát không biết: bốn góc của một tag *cứng với nhau* theo một hình vuông đã biết cạnh, nên mỗi tag chỉ có sáu bậc tự do chứ không phải mười hai. Ràng buộc ấy giảm mạnh số ẩn và cải thiện điều kiện số. Chi phí: vài trăm dòng.

COLMAP [12]. Đường tắt thực dụng: nó là một hệ SfM hoàn chỉnh, nó xử lý được ảnh không có thứ tự, và nó trưởng thành. Nhược điểm là nó được thiết kế cho đặc trưng tự nhiên chứ không cho marker, nên bạn phải đưa các góc tag vào như các điểm đã khớp sẵn — làm được nhưng không phải luồng chính của nó.

tagslam [10]. Được thiết kế đúng cho bài toán này: nó giải factor graph trên nhiều tag và ước lượng cả vị trí tag. Nếu nó chạy được với thiết lập của bạn thì nó là lựa chọn hợp lý nhất. Mức tin C — tiền ẩn — nhưng có mã nguồn và được dùng rộng rãi.

B. Điều còn thiếu sau khi chạy

Đây là câu trả lời cho Q2, và nó là chỗ dễ bỏ sót nhất trong cả chuỗi.

Kết quả BA là tọa độ các góc tag trong một hệ tọa độ *tùy ý* do thuật toán chọn, và nó xác định sai khác một hệ số *tỉ lệ*. Hai thông tin còn thiếu.

Một, tỉ lệ. Ảnh đơn thuần không xác định được kích thước tuyệt đối. Phải neo bằng một khoảng cách đo được — và B/09 mục 5B khuyến nghị đo cạnh của tag *lớn nhất*, vì sai số của phép neo trở thành sai số tỉ lệ của toàn bộ constellation. Đây là chỗ duy nhất trong cả quy trình mà thước cặp còn cần thiết.

Hai, và nó hay bị quên hơn: quan hệ với điểm cập chuẩn. B/12 mục 4 gọi nó là khâu $T_{\text{dock}}^{\text{tag}}$. BA cho bạn các tag ở đâu so với nhau; nó không cho bạn cụm tag ấy ở đâu so với cái lỗ cắm sạc. Cách đo đúng và rẻ nhất, như B/12 mục 4 nêu: đưa robot vào vị trí cập chuẩn rồi đo pose constellation từ đó — pose ấy *định nghĩa* điểm cập chuẩn, không cần thước.

C. Đại lượng phải nhìn trước khi dùng kết quả

Đây là câu trả lời cho Q4: *hiệp phương sai của tọa độ các góc*, mà BA trả về như một sản phẩm phụ.

Cách tự lừa mình nếu bỏ qua nó: bạn thay một con số không biết sai bao nhiêu (từ CAD) bằng một con số khác cũng không biết sai bao nhiêu (từ BA), rồi tin con số thứ hai hơn vì nó “đã được đo”. Nếu BA trả về bất định 2 mm cho một tag — chuyện hoàn toàn có thể khi ảnh ít hoặc góc nhìn nghèo — thì bạn chưa cải thiện gì so với CAD, và bạn đã tiêu một buổi để tin sai.

Phép kiểm cụ thể: so hiệp phương sai BA với dung sai gia công dự kiến. Nếu nó không nhỏ hơn đáng kể thì phải chụp thêm ảnh, đặc biệt là ảnh phủ các tag có bất định lớn từ nhiều góc khác nhau.

Phải kiểm trước khi chuyển sang bước ba: hiệp phương sai đủ nhỏ, và sai số tái chiếu của bài toán hợp nhất không lớn hơn hẳn so với khi giải từng tag riêng (B/09 mục 2).

2.4 Hand-eye

Bước ba.

OpenCV `calibrateHandEye` — cài nhiều phương pháp chọn được: Tsai–Lenz [18], Park–Martin [19], Daniilidis [20], và vài cái khác. Một mẹo chẩn đoán tốt: *chạy với vài phương pháp khác nhau và so kết quả*. Chúng giải cùng một bài toán bằng các đường khác nhau, nên nếu dữ liệu đủ kích

thích thì chúng phải cho kết quả gần nhau. Nếu chúng khác nhau đáng kể thì dữ liệu của bạn không đủ — và đó là chỉ báo rõ nhất cho điều kiện kích thích ở B/12 mục 5B.

`easy_handeye` — wrapper ROS với quy trình thu thập dữ liệu. Tiện, nhưng nó không giải quyết được chuyện điều kiện kích thích: nếu robot của bạn chỉ xoay quanh một trục thì thành phần độ cao vẫn không quan sát được bất kể công cụ nào.

Với robot vi sai, nhớ ba cách xử lý ở B/12 mục 5B — và nhớ rằng cách thứ ba (nhận ra rằng bạn không cần nó) thường là cách đúng.

Phải kiểm sau bước ba: kết quả ổn định qua các tập con dữ liệu, và các phương pháp khác nhau cho kết quả gần nhau.

2.5 Đánh giá: chỗ nên tự viết

Mục này trả lời Q3 và Q5, và nó là mục có nội dung riêng nhất của chương.

`evo` [21] và các công cụ tương tự là công cụ tốt cho việc chúng được thiết kế: so hai quỹ đạo. Chúng có các phép căn chỉnh chuẩn, chúng tính ATE và RPE, và chúng vẽ đồ thị đẹp. Tài liệu tham chiếu về cách dùng chúng cho đúng là [22].

Vì sao chúng không phải công cụ đúng cho nghiệm thu docking — câu trả lời cho Q3: vì bài toán khác về bản chất.

Bài toán quỹ đạo hỏi: hai đường đi này khác nhau bao nhiêu, sau khi đã căn chỉnh tối ưu? Nó có một chuỗi dài các pose và nó quan tâm tới sai số tích lũy.

Bài toán docking hỏi: qua ba mươi lần cập, điểm cuối tản mát bao nhiêu, và nó có lệch so với vị trí đúng không? Nó chỉ quan tâm tới một pose — pose cuối — nhưng qua nhiều lần thử.

Hai câu hỏi ấy cần hai phép tính khác nhau, và phép tính thứ hai đơn giản hơn nhiều: gom các pose cuối, tính trung bình và độ tản mát, và nếu có chuẩn tham chiếu thì so trung bình với nó. Vài chục dòng, và mọi bước đều minh bạch.

Ép `evo` làm việc ấy thì được — coi mỗi lần cập là một “quỹ đạo” một điểm — nhưng bạn phải hiểu rõ nó đang căn chỉnh cái gì, và các phép căn chỉnh của nó được thiết kế cho tình huống khác. Rủi ro không phải là nó không chạy mà là nó chạy và cho một con số mà bạn diễn giải sai.

Nguyên tắc chung rút ra — câu trả lời cho Q5: dùng công cụ có sẵn khi bài toán của bạn đúng là bài toán mà nó được thiết kế cho; tự viết khi bài toán khác, kể cả khi công cụ ấy “làm được”. Ba nhóm đầu của chương thuộc trường hợp thứ nhất — hiệu chuẩn intrinsic, BA, hand-eye đều là đúng bài toán mà công cụ giải. Nhóm thứ tư thuộc trường hợp thứ hai.

Và một lưu ý về cái gì phải đo, lấy từ C/16 mục 4B: đo độ lặp lại của toàn bộ chu trình — lùi ra hẳn rồi cập lại — chứ không đo độ tản mát của pose khi robot đã đứng yên ở vị trí cuối. Hai con số ấy khác nhau nhiều, và chỉ con số thứ nhất có nghĩa với người dùng.

2.6 Giới hạn

Các khẳng định trong chương này chưa được chạy kiểm. Khác với D/17 mục 3B, nơi tôi chạy thật trên OpenCV và phát hiện một cái bẫy, chương này mô tả khả năng của các công cụ từ hiểu biết chung về chúng. Trước khi dựa vào một khẳng định cụ thể — chẳng hạn rằng `calibrateHandEye` có đúng những phương pháp tôi liệt kê — hãy kiểm với tài liệu của phiên bản bạn dùng.

Không có benchmark. Chương không nói công cụ nào cho kết quả chính xác hơn, vì tôi không có dữ liệu so sánh được.

Chốt tháng 9/2026. Cùng cảnh báo như D/17 mục 1.

tagslam ở mức tin C. Tiễn ẩn, chưa qua phản biện; tôi đưa nó vào vì nó có mã nguồn công khai và giải đúng bài toán, nhưng theo `research-rules.md` mục 4 thì nguồn mức C không được đứng một mình làm chỗ dựa.

2.7 Thực hành

Quy trình ba bước, với phép kiểm sau mỗi bước — và các phép kiểm mới là phần đáng giá.

Bước một, intrinsic. Công cụ: OpenCV hoặc Kalibr. Kiểm: bản đồ residual không có cấu trúc; c_x, c_y ổn định qua các tập con.

Bước hai, constellation. Công cụ: Ceres/GTSAM tự viết, hoặc COLMAP, hoặc tagslam. Kiểm: hiệp phương sai đủ nhỏ so với dung sai gia công; và đừng quên neo tỉ lệ cùng khâu T_{dock}^{tag} .

Bước ba, hand-eye. Công cụ: OpenCV `calibrateHandEye`. Kiểm: các phương pháp khác nhau cho kết quả gần nhau; kết quả ổn định qua các tập con.

Và cho đánh giá: tự viết, theo C/16 mục 7.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Ranh giới của chương này với D/17	D/17: thứ chạy <i>mỗi khung</i> (detector, PnP) — thường phải tự cài hoặc tự sửa. Chương này: thứ chạy <i>một lần</i> (hiệu chuẩn, đo đạc, đánh giá) — thường nên dùng đồ có sẵn.
••• Ba bước và ba công cụ	Intrinsic: OpenCV <code>calibrateCamera</code> hoặc Kalibr. Constellation: Ceres/GTSAM tự viết, hoặc COLMAP, hoặc tagslam. Hand-eye: OpenCV <code>calibrateHandEye</code> . Thứ tự <i>bắt buộc</i> theo B/12 mục 2.
••• Vì sao tự viết BA tốt hơn công cụ tổng quát	Vì bạn tận dụng được ràng buộc mà công cụ tổng quát không biết: bốn góc một tag <i>cứng với nhau</i> theo hình vuông đã biết cạnh, nên mỗi tag có 6 bậc tự do chứ không phải 12. Giảm mạnh số ẩn, cải thiện điều kiện số.
••• Hai thứ còn thiếu sau khi chạy BA	Tỉ lệ — ảnh đơn thuần không xác định kích thước tuyệt đối, phải neo bằng cạnh tag <i>lớn nhất</i> . Và khâu T_{dock}^{tag} — BA cho các tag ở đâu so với nhau, không cho cụm ấy ở đâu so với lỗ cấm.
••• Đại lượng phải nhìn sau BA	Hiệp phương sai của tọa độ các góc. Bỏ qua nó là tự lừa mình: thay một con số không biết sai bao nhiêu (CAD) bằng một con số khác cũng không biết sai bao nhiêu (BA), rồi tin cái thứ hai hơn vì nó “đã được đo”.
•• Phép kiểm rẻ nhất cho hand-eye	Chạy với <i>vài phương pháp khác nhau</i> (Tsai–Lenz, Park–Martin, Daniilidis) và so. Chúng giải cùng bài toán bằng các đường khác nhau, nên dữ liệu đủ kích thích thì chúng phải gần nhau. Khác nhau đáng kể $\Rightarrow$ dữ liệu chưa đủ.
••• Vì sao evo không đúng cho docking	Bài toán khác về bản chất. Quỹ đạo: hai đường đi khác nhau bao nhiêu sau khi căn chỉnh tối ưu, quan tâm sai số tích lũy. Docking: qua 30 lần cập, <i>một</i> pose cuối tằn mết bao nhiêu. Hai câu hỏi, hai phép tính.
••• Nguyên tắc chung về chọn công cụ	Dùng đồ có sẵn khi bài toán của bạn <i>đúng là</i> bài toán nó được thiết kế cho; tự viết khi bài toán khác — <i>kể cả khi công cụ ấy “làm được”</i> . Rủi ro không phải nó không chạy mà là nó chạy và bạn diễn giải sai.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
•• Đo cái gì khi nghiệm thu	Độ lặp lại của <i>toàn bộ chu trình</i> — lùi ra hẳn rồi cập lại — chứ không phải độ tần mắt của pose khi robot đã đứng yên ở vị trí cuối. Hai con số khác nhau nhiều (C/16 mục 4B).
•• Phép kiểm sau mỗi bước hiệu chuẩn	Sau intrinsic: bản đồ residual không cấu trúc, c_x, c_y ổn định. Sau constellation: hiệp phương sai đủ nhỏ, residual hợp nhất không lớn hơn hẳn từng tag. Sau hand-eye: các phương pháp cho kết quả gần nhau.
• Chỗ duy nhất thước cặp còn cần	Neo tỉ lệ cho BA — đo cạnh của tag <i>lớn nhất</i> , vì sai số của phép neo thành sai số tỉ lệ toàn cục.
• Mức kiểm chứng của chương này	Các khẳng định <i>chưa được chạy kiểm</i> trong lần soạn này, khác với D/17 mục 3B. Kiểm với tài liệu của phiên bản bạn dùng trước khi dựa vào một khẳng định cụ thể.

••• phải thuộc •• nên nắm • biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
Kalibr [14]	Hiệu chuẩn nhiều mô hình méo, có báo cáo residual theo vị trí ảnh	Công cụ duy nhất mặc định cho bạn phép kiểm quan trọng nhất của A/01 mục 6
Ceres [16] / GTSAM [17]	Tự viết BA với ràng buộc tag cứng — cho kết quả tốt nhất	Vài trăm dòng; đáng nếu constellation là dòng lớn trong ngân sách
COLMAP [12]	SfM hoàn chỉnh, đường tắt cho đo constellation	Thiết kế cho đặc trưng tự nhiên; đưa góc tag vào như điểm đã khớp
OpenCV calibrateHandEye	Nhiều phương pháp chọn được [18]–[20]	Chạy vài phương pháp và so — chỉ báo rõ nhất cho điều kiện kích thích
evo [21]	Đánh giá quỹ đạo với các phép căn chỉnh chuẩn [22]	Tốt cho quỹ đạo, <i>không</i> phải công cụ đúng cho độ lặp lại điểm cuối

MINI PROJECT • Chạy chuỗi ba bước hiệu chuẩn với phép kiểm sau mỗi bước • 3–4 ngày

Mục tiêu	có một chuỗi hiệu chuẩn hoàn chỉnh mà mỗi bước đã được <i>kiểm</i> chứ không chỉ đã <i>chạy</i> — khác biệt quyết định giữa một tệp tham số dùng được và một tệp trông như dùng được.
Dữ liệu	phần cứng thật: camera, target hiệu chuẩn phẳng chất lượng, constellation đã lắp, và robot.
Các bước	bước một, hiệu chuẩn intrinsic với ít nhất 30 ảnh có tư thế nghiêng mạnh và phủ tới bốn góc khung hình; vẽ bản đồ residual và chạy lại với các tập con để kiểm c_x, c_y ; <i>không chuyển bước</i> cho tới khi cả hai phép kiểm đạt. Bước hai, chụp ít nhất 30 ảnh constellation từ nhiều góc, chạy BA, neo tỉ lệ bằng cạnh tag lớn nhất, và lấy hiệp phương sai của kết quả; so nó với dung sai gia công; rồi đo khâu $T_{\text{dock}}^{\text{tag}}$ bằng cách đưa robot vào vị trí cập chuẩn. Bước ba, thu thập cặp pose cho hand-eye, chạy với ba phương pháp khác nhau, so kết quả.
Xong khi	bạn có một tệp cấu hình với ba nhóm tham số, và <i>cho mỗi nhóm</i> một con số bất định kèm theo. Kiểm tra chéo bắt buộc: chạy lại toàn bộ chuỗi từ đầu với một bộ ảnh mới và so hai kết quả — nếu chúng khác nhau nhiều hơn bất định đã ước lượng, thì bất định của bạn đang bị đánh giá thấp, và mọi dòng hiệu chuẩn trong bảng ngân sách của C/15 cũng đang bị đánh giá thấp theo.

Nối với

C/15 nhận ba con số bất định làm ba dòng của bảng; B/12 mini project là phiên bản sâu hơn của phép kiểm ổn định.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Intrinsic: [OpenCV](#) `calibrateCamera` hoặc [Kalibr](#) — cái sau đáng hơn vì nó báo cáo residual theo vị trí ảnh, phép kiểm quan trọng nhất. Constellation: [Ceres](#) hoặc [GTSAM](#) tự viết, hoặc [COLMAP](#), hoặc [tag slam](#). Hand-eye: [OpenCV](#) `calibrateHandEye`. Bước có thể phải tự viết là bước hai, và lý do đáng nói: một công cụ SfM tổng quát không biết rằng bốn góc của một tag *cứng với nhau* theo một hình vuông đã biết cạnh, nên nó xử lý chúng như bốn điểm độc lập — mười hai bậc tự do thay vì sáu. Tự viết cho phép áp ràng buộc ấy, giảm mạnh số ẩn và cải thiện điều kiện số. Với constellation nhỏ thì chênh lệch có thể không đáng, nhưng nếu constellation là dòng lớn trong ngân sách của bạn thì vài trăm dòng code là đầu tư đúng.
- A2. Vì kết quả BA là toạ độ trong một hệ *tuyệt đối* và xác định sai khác một hệ *số tỉ lệ* — ảnh đơn thuần không bao giờ xác định được kích thước tuyệt đối. Thông tin còn thiếu thứ nhất là phép neo tỉ lệ: một khoảng cách đo được bằng thước, và nên đo cạnh của tag *lớn nhất* vì sai số của phép neo trở thành sai số tỉ lệ của toàn bộ constellation. Thông tin còn thiếu thứ hai, và nó hay bị quên hơn: quan hệ giữa hệ constellation và *điểm cập chuẩn thật* — khâu T_{dock}^{tag} của B/12 mục 4. BA cho bạn biết các tag ở đâu so với nhau; nó hoàn toàn không biết cái lỗ cắm sạc ở đâu. Cách đo khâu ấy rẻ và đẹp: đưa robot vào vị trí cập đúng rồi đo pose constellation từ đó — pose ấy *định nghĩa* điểm cập chuẩn, và không cần thước nào cả.
- A3. Vì bài toán khác về bản chất chứ không chỉ về quy mô. [evo](#) được thiết kế cho bài toán *quỹ đạo*: so hai đường đi dài sau khi căn chỉnh tối ưu, với các chỉ số ATE và RPE quan tâm tới sai số tích lũy. Bài toán docking hỏi một câu khác hẳn: qua ba mươi lần cập độc lập, *một* pose — pose cuối — tằn mát bao nhiêu, và nó có lệch so với vị trí đúng không? Đó là một bài toán thống kê trên nhiều lần thử ở một điểm, không phải một bài toán so sánh hai chuỗi. Phép tính đúng đơn giản hơn nhiều: gom các pose cuối, tính trung bình và độ tằn mát, so trung bình với chuẩn tham chiếu nếu có. Ép [evo](#) làm việc ấy thì chạy được, nhưng rủi ro không phải là nó không chạy — rủi ro là nó chạy và cho một con số mà bạn diễn giải sai, vì bạn không biết chính xác nó đã căn chỉnh cái gì.
- A4. Hiệp phương sai của toạ độ các góc, mà BA trả về như một sản phẩm phụ. Cách tự lừa mình nếu bỏ qua nó: bạn bắt đầu với một con số từ CAD mà bạn không biết sai bao nhiêu, bạn chạy BA và được một con số khác mà bạn *cũng* không biết sai bao nhiêu, rồi bạn tin con số thứ hai hơn vì nó “đã được đo”. Nhưng nếu BA trả về bất định 2 mm cho một tag — hoàn toàn có thể khi số ảnh ít, góc nhìn nghèo, hoặc intrinsic chưa tốt — thì bạn chưa cải thiện gì so với CAD, và bạn vừa tiêu một buổi để có được sự tự tin sai chỗ. Phép kiểm cụ thể: so hiệp phương sai BA với dung sai gia công dự kiến; nếu nó không nhỏ hơn đáng kể thì phải chụp thêm ảnh, đặc biệt là ảnh phủ các tag có bất định lớn từ nhiều góc khác nhau.
- A5. Nhóm *đánh giá* nên tự viết; ba nhóm còn lại — hiệu chuẩn intrinsic, đo constellation, hand-eye — nên dùng đồ có sẵn. Nguyên tắc chung: *dùng công cụ có sẵn khi bài toán của bạn đúng là bài toán mà nó được thiết kế cho, và tự viết khi bài toán khác — kể cả khi công cụ ấy “làm được”*. Ba nhóm đầu thuộc trường hợp thứ nhất: hiệu chuẩn camera là đúng bài toán mà `calibrateCamera` giải, BA là đúng bài toán mà `Ceres` giải. Nhóm thứ tư thuộc trường hợp thứ hai: [evo](#) giải bài toán quỹ đạo, còn bạn có bài toán độ lặp lại một điểm. Điều làm nguyên tắc này đáng nhớ là chỗ “*kể cả khi làm được*”: một công cụ ép vào bài toán sai vẫn cho ra số, và số ấy trông hợp lệ — đó mới là rủi ro, chứ không phải chuyện nó báo lỗi.

NÓI LẠI THẬT ĐƠN GIẢN

Chương trước nói về mấy thứ chạy liên tục khi robot đang làm việc. Chương này nói về mấy thứ chạy đúng một lần rồi cho ra một tệp con số.

Có ba việc như vậy, và chúng phải làm đúng thứ tự: đo cái camera trước, rồi đo xem mấy tấm tag thật sự nằm đâu, rồi cuối cùng đo xem cái camera được bắt ở chỗ nào trên robot. Làm ngược thì sai số của bước trước chui vào bước sau và không còn tách ra được.

Điều quan trọng nhất của chương không phải tên công cụ mà là: *sau mỗi bước phải kiểm, chứ không phải chạy xong là tin*. Và phép kiểm thường rất đơn giản — chạy lại với một nửa dữ liệu rồi xem kết quả có đổi không. Con số nào giữ nguyên thì tin được; con số nào nhảy thì nó chưa được xác định, dù phần mềm có in nó ra với năm chữ số thập phân.

Riêng bước đo mấy tấm tag có một cái bẫy đáng nhớ. Bạn chụp mấy chục tấm ảnh, chạy chương trình, và nó cho ra một mô hình ba chiều rất đẹp. Nhưng mô hình ấy chưa dùng được, vì nó thiếu hai thứ. Thứ nhất: ảnh không biết kích thước thật — một cái dock nhỏ chụp gần và một cái dock lớn chụp xa cho ảnh giống hệt nhau. Phải đo một cái cạnh bằng thước để neo lại, và đây là chỗ duy nhất trong cả quy trình mà cây thước còn cần thiết. Thứ hai, và hay quên hơn: chương trình cho biết mấy tấm tag nằm đâu *so với nhau*, nhưng nó hoàn toàn không biết cái lỗ cắm sạc nằm đâu. Cách đo cái đó thì đẹp: cứ đẩy robot vào cắm cho đúng, rồi chụp một tấm — thế là xong, không cần đo gì cả.

Cuối cùng là chuyện chọn công cụ, và có một nguyên tắc đáng nhớ. Dùng đồ có sẵn khi việc của bạn đúng là việc mà người ta làm ra nó để giải quyết. Tự viết khi việc của bạn khác — kể cả khi cái công cụ kia “cũng làm được”. Chỗ nguy hiểm không phải là công cụ báo lỗi; chỗ nguy hiểm là nó chạy trơn tru và cho ra một con số mà bạn hiểu nhầm.

Ví dụ cụ thể: có những công cụ rất tốt để so hai đường đi của robot. Việc của bạn thì khác — bạn cần biết qua ba mươi lần cập dock, cái điểm cuối tảo mất bao nhiêu. Ép cái công cụ kia làm việc này thì được, nhưng viết ba chục dòng tự tính thì đúng hơn và bạn hiểu rõ mình đang tính gì.

Chỗ không rút gọn được: không có cách nào biết trước một công cụ có phù hợp hay không ngoài việc hiểu rõ nó được thiết kế để giải bài toán nào. Tài liệu thường nói nó *làm được gì*, ít khi nói nó *được thiết kế cho gì* — và hai điều đó không giống nhau.

Một câu: *Chạy xong chưa phải xong — chạy lại với nửa dữ liệu rồi xem con số nào nhảy, đó mới là kiểm.*

2.8 Điều gì chứng minh cách hiểu này sai

Chương này có mức kiểm chứng *thấp hơn D/17*, và mục 6 đã ghi rõ lý do: tôi không chạy các công cụ trong lần soạn này. Vì vậy mục này liệt kê những khẳng định đáng nghi nhất.

Khẳng định có hậu quả lớn nhất — *tự viết BA với ràng buộc tag cứng cho kết quả tốt hơn dùng một công cụ SfM tổng quát* — là suy luận của tôi từ việc đếm bậc tự do (sáu thay vì mười hai cho mỗi tag). Nó bị bác bỏ nếu một so sánh thật cho thấy COLMAP với các góc tag đưa vào như điểm đã khớp đạt cùng độ chính xác. Điều đó hoàn toàn có thể khi số ảnh lớn, vì khi ấy thông tin dư thừa bù được cho ràng buộc bị bỏ qua — và nếu vậy thì lời khuyên “tự viết” là lời khuyên tốn công vô ích. Đây là mệnh đề đáng kiểm tra trước khi bỏ ra vài trăm dòng.

Khẳng định thứ hai — *chạy hand-eye với nhiều phương pháp rồi so là chỉ báo cho điều kiện kích thích* — dựa trên lập luận rằng các phương pháp giải cùng bài toán bằng các đường khác nhau nên phải hội tụ khi dữ liệu đủ. Nó bị bác bỏ nếu hai phương pháp cho kết quả khác nhau *ngay cả khi* dữ liệu đủ kích thích, chẳng hạn vì chúng xử lý nhiễu theo những cách khác nhau. Trong trường hợp ấy phép thử cho báo động giả, và phép thử đúng phải là chạy lại với các tập con dữ liệu.

Khẳng định thứ ba — *công cụ đánh giá quỹ đạo không phù hợp cho nghiệm thu docking* — là một mệnh đề về sự phù hợp của công cụ với bài toán, không phải một mệnh đề thực nghiệm. Nó “bị bác bỏ” nếu ai đó chỉ ra một cách dùng *evo* cho bài toán độ lặp lại điểm cuối mà vừa đúng vừa đơn giản hơn vài chục dòng tự viết. Tôi không biết cách nào, nhưng tôi cũng không khẳng định là không có.

2.9 Kết nối

Nối lên trên. B/12-calibration-cho-docking là chương lý thuyết tương ứng: nó nói *vì sao* ba bước phải theo thứ tự ấy và phải kiểm gì; chương này nói dùng công cụ nào. B/09-multi-marker-constellation mục 5 là nguồn của mục 3.

Nối ngang. D/17-thu-vien-va-detector là chương anh em, chia theo ranh giới mỗi-khung so với một-lần.

Nối xuống. C/15-ngan-sach-sai-so nhận ba con số bắt định từ mini project của chương này làm ba dòng của bảng. C/16-danh-gia-va-nghiem-thu mục 7 là nơi mục 5 dẫn tới: quy trình nghiệm thu tự viết.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Nêu ba bước hiệu chuẩn, công cụ cho từng bước, và phép kiểm bắt buộc sau mỗi bước.
2. Vì sao tự viết BA có thể tốt hơn dùng một công cụ SfM tổng quát? Nêu ràng buộc mà công cụ tổng quát không biết.
3. Sau khi BA chạy xong, bạn còn thiếu hai thông tin. Nêu cả hai, và nêu cách đo thông tin thứ hai mà không cần thước.
4. Vì sao bỏ qua hiệp phương sai của kết quả BA là tự lừa mình? Mô tả cơ chế.
5. Nêu phép kiểm rẻ nhất cho điều kiện kích thích của hand-eye.
6. Vì sao công cụ đánh giá quỹ đạo không phải công cụ đúng cho nghiệm thu docking, và rủi ro cụ thể của việc ép nó là gì?
7. Phát biểu nguyên tắc chung về khi nào dùng đồ có sẵn và khi nào tự viết.

Nguồn

- [1] G. Terzakis and M. Lourakis, “A Consistently Fast and Globally Optimal Solution to the Perspective-n-Point Problem,” in *European Conference on Computer Vision (ECCV)*, mức tin B; SQPnP, có trong OpenCV, 2020.
- [2] V. Lepetit, F. Moreno-Noguer, and P. Fua, “EPnP: An Accurate $O(n)$ Solution to the PnP Problem,” *International Journal of Computer Vision*, vol. 81, no. 2, pp. 155–166, 2009, mức tin A.
- [3] B. Benligray, C. Topal, and C. Akinlar, “STag: A Stable Fiducial Marker System,” in *Image and Vision Computing*, mức tin B; dùng đường tròn ngoại tiếp để ổn định pose, vol. 89, 2019, pp. 158–169.
- [4] G. Yu, Y. Hu, and J. Dai, “TopoTag: A Robust and Scalable Topological Fiducial Marker System,” in *IEEE Transactions on Visualization and Computer Graphics*, mức tin B, 2021.
- [5] J. DeGol, T. Bretl, and D. Hoiem, “ChromaTag: A Colored Marker and Fast Detection Algorithm,” in *IEEE International Conference on Computer Vision (ICCV)*, mức tin B, 2017.
- [6] L. Calvet, P. Gurdjos, C. Griwodz, et al., “Detection and Accurate Localization of Circular Fiducials under Highly Challenging Conditions,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; CCTag, vòng tròn đồng tâm định vị chính xác, 2016.
- [7] T. Krajník, M. Nitsche, J. Faigl, et al., “A Practical Multirobot Localization System,” *Journal of Intelligent & Robotic Systems*, vol. 76, pp. 539–562, 2014, mức tin B; WhyCon, marker tròn đồng tâm.
- [8] Z. Zhang, Y. Hu, G. Yu, et al., “DeepTag: A General Framework for Fiducial Marker Design and Detection,” in *IEEE Transactions on Pattern Analysis and Machine Intelligence*, mức tin B, 2023.
- [9] R. Berral-Soler, F. J. Madrid-Cuevas, R. Muñoz-Salinas, et al., “DeepArUco: Marker Detection and Classification in Challenging Lighting Conditions,” *IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2023, mức tin C; tiền ấn/-workshop, dùng làm gợi ý.
- [10] B. Pfrommer and K. Daniilidis, “TagSLAM: Robust SLAM with Fiducial Markers,” in *arXiv preprint arXiv:1910.00679*, mức tin C; tiền ấn nhưng có mã nguồn công khai và được dùng rộng rãi — dùng làm gợi ý về kiến trúc, không làm bằng chứng số, 2019.
- [11] É. Marchand, F. Spindler, and F. Chaumette, “ViSP for Visual Servoing: A Generic Software Platform with a Wide Class of Robot Control Skills,” *IEEE Robotics & Automation Magazine*, vol. 12, no. 4, pp. 40–52, 2005, mức tin A.
- [12] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; COLMAP, 2016.
- [13] Z. Zhang, “A Flexible New Technique for Camera Calibration,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 11, pp. 1330–1334, 2000, mức tin A; phương pháp hiệu chuẩn từ target phẳng mà mọi hộp công cụ đang dùng.
- [14] P. Furgale, J. Rehder, and R. Siegwart, “Unified Temporal and Spatial Calibration for Multi-Sensor Systems,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; nền của Kalibr, 2013.

- [15] H. Ha, M. Perdoch, H. Alismail, et al., “Deltille Grids for Geometric Camera Calibration,” in *IEEE International Conference on Computer Vision (ICCV)*, mức tin A; lưới tam giác và tính chỉnh saddle point bậc hai, chính xác hơn checkerboard, 2017.
- [16] S. Agarwal, K. Mierle, et al., “Ceres Solver,” <http://ceres-solver.org>, 2012, mức tin A; phần mềm, không phải bài báo.
- [17] F. Dellaert and M. Kaess, “Factor Graphs for Robot Perception,” *Foundations and Trends in Robotics*, vol. 6, no. 1-2, pp. 1–139, 2017, mức tin A.
- [18] R. Y. Tsai and R. K. Lenz, “A New Technique for Fully Autonomous and Efficient 3D Robotics Hand/Eye Calibration,” *IEEE Transactions on Robotics and Automation*, vol. 5, no. 3, pp. 345–358, 1989, mức tin A; lời giải kinh điển cho $AX=XB$.
- [19] F. C. Park and B. J. Martin, “Robot Sensor Calibration: Solving $AX=XB$ on the Euclidean Group,” *IEEE Transactions on Robotics and Automation*, vol. 10, no. 5, pp. 717–721, 1994, mức tin A.
- [20] K. Daniilidis, “Hand-Eye Calibration Using Dual Quaternions,” *The International Journal of Robotics Research*, vol. 18, no. 3, pp. 286–298, 1999, mức tin A; giải đồng thời xoay và tịnh tiến.
- [21] M. Grupp, “evo: Python Package for the Evaluation of Odometry and SLAM,” <https://github.com/MichaelGrupp/evo>, 2017, mức tin B; phần mềm.
- [22] Z. Zhang and D. Scaramuzza, “A Tutorial on Quantitative Trajectory Evaluation for Visual(-Inertial) Odometry,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; căn chỉnh quỹ đạo trước khi tính sai số, và vì sao chọn sai phép căn chỉnh làm con số vô nghĩa, 2018.

Nguồn

- [1] G. Terzakis and M. Lourakis, “A Consistently Fast and Globally Optimal Solution to the Perspective-n-Point Problem,” in *European Conference on Computer Vision (ECCV)*, mức tin B; SQPnP, có trong OpenCV, 2020.
- [2] V. Lepetit, F. Moreno-Noguer, and P. Fua, “EPnP: An Accurate $O(n)$ Solution to the PnP Problem,” *International Journal of Computer Vision*, vol. 81, no. 2, pp. 155–166, 2009, mức tin A.
- [3] B. Benligray, C. Topal, and C. Akinlar, “STag: A Stable Fiducial Marker System,” in *Image and Vision Computing*, mức tin B; dùng đường tròn ngoại tiếp để ổn định pose, vol. 89, 2019, pp. 158–169.
- [4] G. Yu, Y. Hu, and J. Dai, “TopoTag: A Robust and Scalable Topological Fiducial Marker System,” in *IEEE Transactions on Visualization and Computer Graphics*, mức tin B, 2021.
- [5] J. DeGol, T. Bretl, and D. Hoiem, “ChromaTag: A Colored Marker and Fast Detection Algorithm,” in *IEEE International Conference on Computer Vision (ICCV)*, mức tin B, 2017.
- [6] L. Calvet, P. Gurdjos, C. Griwodz, et al., “Detection and Accurate Localization of Circular Fiducials under Highly Challenging Conditions,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; CCTag, vòng tròn đồng tâm định vị chính xác, 2016.
- [7] T. Krajník, M. Nitsche, J. Faigl, et al., “A Practical Multirobot Localization System,” *Journal of Intelligent & Robotic Systems*, vol. 76, pp. 539–562, 2014, mức tin B; WhyCon, marker tròn đồng tâm.
- [8] Z. Zhang, Y. Hu, G. Yu, et al., “DeepTag: A General Framework for Fiducial Marker Design and Detection,” in *IEEE Transactions on Pattern Analysis and Machine Intelligence*, mức tin B, 2023.
- [9] R. Berral-Soler, F. J. Madrid-Cuevas, R. Muñoz-Salinas, et al., “DeepArUco: Marker Detection and Classification in Challenging Lighting Conditions,” *IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2023, mức tin C; tiền ấn/-workshop, dùng làm gợi ý.
- [10] B. Pfrommer and K. Daniilidis, “TagSLAM: Robust SLAM with Fiducial Markers,” in *arXiv preprint arXiv:1910.00679*, mức tin C; tiền ấn nhưng có mã nguồn công khai và được dùng rộng rãi — dùng làm gợi ý về kiến trúc, không làm bằng chứng số, 2019.
- [11] É. Marchand, F. Spindler, and F. Chaumette, “ViSP for Visual Servoing: A Generic Software Platform with a Wide Class of Robot Control Skills,” *IEEE Robotics & Automation Magazine*, vol. 12, no. 4, pp. 40–52, 2005, mức tin A.
- [12] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; COLMAP, 2016.
- [13] Z. Zhang, “A Flexible New Technique for Camera Calibration,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 11, pp. 1330–1334, 2000, mức tin A; phương pháp hiệu chuẩn từ target phẳng mà mọi hộp công cụ đang dùng.
- [14] P. Furgale, J. Rehder, and R. Siegwart, “Unified Temporal and Spatial Calibration for Multi-Sensor Systems,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; nền của Kalibr, 2013.

- [15] H. Ha, M. Perdoch, H. Alismail, et al., “Deltille Grids for Geometric Camera Calibration,” in *IEEE International Conference on Computer Vision (ICCV)*, mức tin A; lưới tam giác và tính chỉnh saddle point bậc hai, chính xác hơn checkerboard, 2017.
- [16] S. Agarwal, K. Mierle, et al., “Ceres Solver,” <http://ceres-solver.org>, 2012, mức tin A; phần mềm, không phải bài báo.
- [17] F. Dellaert and M. Kaess, “Factor Graphs for Robot Perception,” *Foundations and Trends in Robotics*, vol. 6, no. 1-2, pp. 1–139, 2017, mức tin A.
- [18] R. Y. Tsai and R. K. Lenz, “A New Technique for Fully Autonomous and Efficient 3D Robotics Hand/Eye Calibration,” *IEEE Transactions on Robotics and Automation*, vol. 5, no. 3, pp. 345–358, 1989, mức tin A; lời giải kinh điển cho $AX=XB$.
- [19] F. C. Park and B. J. Martin, “Robot Sensor Calibration: Solving $AX=XB$ on the Euclidean Group,” *IEEE Transactions on Robotics and Automation*, vol. 10, no. 5, pp. 717–721, 1994, mức tin A.
- [20] K. Daniilidis, “Hand-Eye Calibration Using Dual Quaternions,” *The International Journal of Robotics Research*, vol. 18, no. 3, pp. 286–298, 1999, mức tin A; giải đồng thời xoay và tịnh tiến.
- [21] M. Grupp, “evo: Python Package for the Evaluation of Odometry and SLAM,” <https://github.com/MichaelGrupp/evo>, 2017, mức tin B; phần mềm.
- [22] Z. Zhang and D. Scaramuzza, “A Tutorial on Quantitative Trajectory Evaluation for Visual(-Inertial) Odometry,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; căn chỉnh quỹ đạo trước khi tính sai số, và vì sao chọn sai phép căn chỉnh làm con số vô nghĩa, 2018.